AZURE ENTERPRISE PLATFORM LAB

Architecture, infrastructure as code, CI/CD and operations guide

PROJECT DOCUMENTATION | Version 1.0 | 30 October 2026

PROJECT	IMPLEMENTATION
Cloud platform	Microsoft Azure
Infrastructure as Code	Terraform with reusable modules and Azure Storage remote state
Container platform	Azure Kubernetes Service (AKS)
Container registry	Azure Container Registry (ACR)
Delivery automation	Azure DevOps YAML pipelines
Source repository	https://github.com/shareq67/azure-enterprise-platform-lab

Purpose

This guide describes the lab's implemented architecture, Terraform-managed infrastructure, application delivery process, infrastructure delivery process, validation evidence, operating procedures and known limitations. It is written for engineers reviewing the repository, reproducing the lab, or preparing to explain the implementation.

Scope and accuracy

This is a hands-on learning and demonstration environment. It is not represented as a fully production-certified Azure landing zone. Implemented components and observed pipeline results are distinguished from recommended future improvements. Environment-specific identifiers, public IPs, subscription IDs, credentials and personal account details are intentionally excluded from this public document.

1. Executive summary

The Azure Enterprise Platform Lab demonstrates a repeatable way to provision and manage Azure infrastructure with Terraform, deploy a containerized application to AKS, and automate both workflows through separate Azure DevOps pipelines.

Capability	Implementation	Evidence / status
Infrastructure as Code	Terraform root configuration and modules	Format, validate and plan completed; saved plan applied
Remote state	Azure Storage backend with Azure AD authentication	Backend initialized and state lock acquired/released in pipeline logs
Networking	Hub and workload VNets, subnets, NSGs and VNet peering	Terraform-managed resources refreshed successfully
Container platform	AKS cluster and ACR	AKS provisioning state verified as Succeeded
Application delivery	Unit tests, Docker build, ACR push, AKS rollout and health check	Pipeline implementation present; run the app pipeline to validate latest execution
Infrastructure delivery	Terraform plan artifact, environment approval, apply and verification	Successful run; 0 added, 0 changed, 0 destroyed
Observability foundation	Log Analytics workspace	Workspace exists; AKS telemetry integration remains follow-up

Key outcome

Infrastructure pipeline run: the approved saved plan completed successfully with **0 resources added, 0 changed and 0 destroyed**. The AKS verification command returned the cluster provisioning state as **Succeeded**. This confirms that the deployed infrastructure matched the Terraform configuration at the time of that run; it does not mean the run created new resources.

Repository

Source code and YAML pipeline definitions: github.com/shareq67/azure-enterprise-platform-lab. The default branch used in the recorded pipeline run was **master**.

2. Solution architecture

The lab combines a segmented Azure network foundation, an AKS-hosted container application, ACR as the image registry, Log Analytics as an observability workspace foundation, and Azure DevOps pipelines for delivery automation.

Logical architecture

Consumer Browser / API client	**Application endpoint** Public service endpoint Health route: /health	**AKS** Deployment: aep-lab-app Pod runs container image
Azure DevOps – App pipeline Tests → Docker build → ACR push → AKS deploy → health check	**Azure Container Registry** Stores versioned app image	**Log Analytics** Workspace provisioned AKS integration is follow-up
Azure DevOps – Infra pipeline fmt → validate → plan artifact → approval → apply → verify	**Terraform** Root configuration and modules	**Azure Storage backend** Remote Terraform state and locking

Network topology

- **Hub VNet:** centralized network segment with management and shared-services subnets.
- **Workload VNet:** workload network segment with AKS node and private-endpoint subnets.
- **VNet peering:** hub-to-workload and workload-to-hub peerings provide connectivity between the two virtual networks.
- **Network security:** management and workload NSGs are provisioned, with subnet-to-NSG associations and diagnostic settings for the NSGs.

Important architecture boundary

Do not assume that Azure Firewall, Bastion, Application Gateway/WAF, private AKS API, private endpoints for every service, or a multi-region disaster recovery design are deployed just because they appear in a generic enterprise reference architecture. This document only claims the components evidenced in the current lab configuration and pipeline logs.

3. Azure resource and configuration inventory

The following names are the lab resource names evidenced in the project context. They are included to make the documentation useful for repository readers; subscription IDs, IP addresses and backend storage account identifiers are omitted.

Resource / component	Name / configuration	Purpose
Network resource group	rg-aep-lab-network	Network and shared platform resources
AKS resource group	rg-aep-lab-aks	Kubernetes cluster resources
Hub VNet	vnet-aep-lab-hub	Management and shared-services network
Workload VNet	vnet-aep-lab-workload	AKS and private-endpoint subnet segments
Subnets	snet-management; snet-shared-services; snet-aks-nodes; snet-private-endpoints	Subnet segmentation by workload role
Network security groups	nsg-aep-lab-management; nsg-aep-lab-workload	Network traffic filtering
AKS cluster	aks-aep-lab	Managed Kubernetes platform
AKS deployment	aep-lab-app	Application workload deployment
Container registry	shareqaepacr2026	Stores application container images
Log Analytics workspace	law-aep-lab-centralindia	Central log and telemetry workspace foundation
Terraform state resource group	rg-aep-lab-tfstate	Hosts remote state storage

AKS lab configuration (observed)

- Node pool: one node in the lab configuration; VM size Standard_D2s_v3.
- AKS SKU: Free tier; cluster identity: SystemAssigned.
- Network mode: Azure network overlay; service CIDR: 10.30.0.0/16.
- Application deployment: aep-lab-app; health endpoint: /health.

Cost and lifecycle note

This lab was built with a limited Azure credit/budget and a time-limited subscription. Stop or delete resources that are not needed, review current Azure pricing before enabling optional services, and confirm subscription status before relying on the environment for a long-running demo. Do not commit billing exports or secrets to the public repository.

4. Infrastructure as Code – Terraform

Terraform is used to describe Azure resources declaratively and to keep infrastructure changes reviewable in source control. The lab uses a root configuration that composes focused modules.

Module responsibilities

Module	Responsibility
networking	Resource groups, VNets, subnets, NSGs, peerings and NSG associations
aks	AKS resource group and managed Kubernetes cluster
acr	Azure Container Registry
monitoring	Log Analytics workspace
diagnostics	Diagnostic settings for the configured network security groups

Typical Terraform workflow

```
terraform fmt -recursive infra<br/>terraform -chdir=infra/environments/lab init<br/>terraform
-chdir=infra/environments/lab validate<br/>terraform -chdir=infra/environments/lab plan<br/>terraform
-chdir=infra/environments/lab apply
```

Remote state design

- State is stored remotely in Azure Storage rather than committed to Git.
- The backend uses Azure AD authentication (`use_azuread_auth=true`) in the pipeline.
- The pipeline initializes the same backend configuration for plan and apply.
- Terraform state locking is used to reduce the risk of concurrent state operations.
- The pipeline publishes a saved binary plan as a pipeline artifact and applies that exact artifact after approval.

Operational best practices

- Never commit `terraform.tfstate`, `*.tfstate.backup`, plan files, credentials or local `.terraform` directories.
- Keep `.terraform.lock.hcl` in source control to make provider selection reproducible; review and commit intentional provider lock-file changes.
- Use separate variable files or workspaces only when environment separation is intentionally designed and documented.
- Review every plan for unexpected create, update, replace or destroy actions before approving apply.
- Use least-privilege role assignments for the pipeline identity and protect the state storage account.

Version alignment

The pipeline configuration documented for this run installs Terraform CLI **1.13.3**. The AzureRM provider initialized in the apply logs was **4.81.0**. Keep local development and CI versions aligned and update them deliberately.

5. CI/CD design

Two independent YAML pipelines are used so that application releases do not implicitly run infrastructure provisioning, and infrastructure changes do not rebuild or redeploy the application.

5.1 Infrastructure pipeline – azure-pipelines.yml

Stage	Actions	Control
TerraformPlan	Install Terraform; fmt check; init/validate; authenticate to Azure; initialize remote backend; create plan; publish plan artifact.	Read/plan workflow; no resource changes.
TerraformApply	Download saved plan; initialize same backend; apply saved plan; query AKS provisioning state.	Azure DevOps environment approval/check before deployment job runs.

Observed pipeline result

- Plan stage: successful; Terraform reported no differences between deployed resources and configuration.
- Approval: environment check and approval passed.
- Apply stage: `Apply complete! Resources: 0 added, 0 changed, 0 destroyed.`
- Verification: AKS cluster `aks-aep-lab` returned state `Succeeded`.

5.2 Application pipeline – azure-pipelines-app.yml

Stage	Actions
Test	Set up Python 3.12; install application dependencies; run Python unittest discovery under tests/.
BuildAndPush	Authenticate using Azure service connection; log in to ACR; build image tagged with the Azure DevOps Build ID; push image to ACR.
Deploy	Get AKS credentials; update deployment image; wait for rollout; inspect pods/services; call application /health endpoint.

Pipeline triggers

- Infrastructure pipeline: master branch changes under `infra/**` or changes to `azure-pipelines.yml`.
- Application pipeline: master branch changes under `app/**`, `tests/**`, `Dockerfile` or `azure-pipelines-app.yml`.

Service connection

Both pipelines reference the Azure Resource Manager service connection named `mi-aep-lab-pipeline`. The connection must be authorized for the relevant pipeline and granted only the Azure roles/scopes required for its tasks. Do not publish credentials or service-principal secrets in YAML.

6. Application request and release flow

Application release flow

```
Developer pushes application change to master<br/> ↓<br/>Application pipeline trigger (app/, tests/,
Dockerfile or YAML)<br/> ↓<br/>Python dependencies installed + unit tests executed<br/> ↓<br/>Docker
image built with Build.BuildId tag<br/> ↓<br/>Image pushed to Azure Container Registry<br/>
↓<br/>Pipeline obtains AKS credentials<br/> ↓<br/>Deployment image updated (kubectl set image)<br/>
↓<br/>Kubernetes rollout status checked<br/> ↓<br/>Pods / Services inspected + /health endpoint checked
```

Runtime request flow (conceptual)

- A client sends an HTTP request to the application’s published endpoint.
- The Kubernetes Service / ingress routing configured for the workload directs traffic to the application pod. Confirm the actual Service type and ingress resources in the live cluster before documenting a more detailed network path.
- The application container handles the request and returns the response. The `/health` route is used by the pipeline as a post-deployment health check.
- The AKS node pulls the required container image from ACR using the cluster’s configured registry access.

Do not overstate

The current pipeline uses `kubectl set image` against an existing Deployment. This is a straightforward demonstration release mechanism, not a complete GitOps reconciliation model. The documented evidence does not establish a production ingress controller, WAF, API gateway, canary release strategy, blue/green deployment or multi-region failover.

Operational checks

Check	Command / action	Expected signal
Cluster nodes	kubectl get nodes	Node status Ready
Pods	kubectl get pods -o wide	Expected replicas Running / Ready
Deployment	kubectl rollout status deployment/aep-lab-app	Rollout successfully completed
Application logs	kubectl logs deployment/aep-lab-app	No recurring errors or crash loops
Health endpoint	curl -fsS https:///health	Successful health response; use actual endpoint and scheme

7. Monitoring, security and operational controls

Monitoring status

- A Log Analytics workspace named `law-aep-lab-centralindia` is provisioned.
- Diagnostic settings are configured for the management and workload NSGs.
- The reviewed configuration showed AKS Azure Monitor metrics disabled, OMS agent configuration absent, and no AKS diagnostic settings configured. Therefore, the workspace should be treated as a monitoring foundation, not proof of end-to-end AKS observability.

Recommended monitoring follow-up

- Enable the appropriate AKS monitoring integration for the selected Azure Monitor architecture and verify data ingestion.
- Configure alerts for node/pod health, restart counts, CPU/memory pressure, failed deployments and application health.
- Define log retention and ingestion budget; validate actual daily ingestion before increasing retention or enabling verbose collection.
- Document alert ownership, severity, response steps and escalation contacts.

Security practices

- Use managed identities or workload identity where supported; avoid static secrets in repository variables or YAML.
- Use least privilege for the Azure service connection and separate read/plan permissions from apply permissions where practical.
- Restrict who can approve the infrastructure environment and who can edit protected pipeline YAML.
- Protect remote state storage with restricted network access where practical, encryption at rest, soft delete/versioning and tightly scoped data-plane roles.
- Pin Terraform and provider versions; review provider upgrades and lock-file diffs.
- Scan container images and dependencies; use non-root containers and minimal base images where supported by the app.
- Keep public repository content free of subscription IDs, tenant IDs, private IP plans if sensitive, public endpoint addresses, state files, credentials and exported logs containing identifiers.

Known limitations / follow-up backlog

Priority	Follow-up item	Reason / completion evidence
High	Enable and validate AKS monitoring integration	Workspace exists, but AKS telemetry integration was not enabled in the reviewed configuration.
High	Review app pipeline health-check endpoint handling	Avoid hard-coded public IP; discover endpoint or use a controlled pipeline variable and validate TLS.
Medium	Add Terraform security / static analysis checks	Add tools such as tfsec or Checkov after selecting policy and handling exceptions.
Medium	Add Kubernetes security and policy controls	Review Pod Security Standards, RBAC, network policies and image scanning.
Medium	Test backup and disaster recovery procedures	Document recovery objectives and perform a restore exercise; do not claim DR until tested.
Medium	Add automated infrastructure verification	Extend checks beyond AKS provisioning state to key network, registry and workspace resources.

8. Reproducing the lab and troubleshooting

Prerequisites

- An Azure subscription with sufficient quota and budget for AKS, networking, registry and monitoring resources.
- Azure CLI, Terraform CLI compatible with the pipeline version, kubectl, Docker and Git.
- An Azure DevOps project connected to the GitHub repository and authorized service connection.
- An Azure Storage backend already bootstrapped and accessible to the pipeline identity.
- An Azure DevOps environment named `aep-lab-infra-approval` with approvals/checks configured.

Safe infrastructure workflow

```
git checkout master<br/>git pull origin master<br/>terraform fmt -check -recursive infra<br/>terraform
-chdir=infra/environments/lab init<br/>terraform -chdir=infra/environments/lab validate<br/>terraform
-chdir=infra/environments/lab plan
```

For CI/CD, push a reviewed change to the relevant trigger path. Inspect the Terraform plan artifact and all proposed replacements/deletions. Approve only when the change is understood and authorized. Never approve a plan solely because the pipeline is green.

Common troubleshooting

Symptom	Likely area to inspect
Environment not found / unauthorized	Create the exact Azure DevOps Environment name used by YAML; authorize pipeline access; configure approvals/checks.
Terraform backend authentication failure	Service connection identity, Azure Storage data-plane role, backend account/container/key and Azure AD auth configuration.
Terraform plan shows unexpected replacement	Review plan details, provider version/lock file, resource drift, immutable property changes and state mapping. Do not approve until understood.
ACR push denied	Service connection permissions, registry role, ACR login and image repository/tag.
AKS credentials / kubectl denied	AKS cluster access, Kubernetes authorization/RBAC, service connection permissions and agent network reachability.
Rollout timeout	kubectl describe deployment/pod, events, image pull errors, readiness/liveness probes and application logs.
Health check fails	Service endpoint, routing, NSG rules, app health route, DNS/TLS and endpoint reachability from agent.

Evidence from the recorded infrastructure pipeline run

The following redacted excerpt captures the successful saved-plan apply result and AKS verification. Account identity and subscription-specific identifiers are intentionally excluded.

Terraform apply result

```
2s
2s
74 rerun this command to reinitialize your working directory. If you forget, other
75 commands will detect it and remind you to do so if necessary.
76 Applying the approved saved plan...
77
```

AKS verification result

```
93 Terraform Apply completed.
94 Verifying AKS cluster...
95 Name         State
96 -----------  ---------
97 aks-aep-lab  Succeeded
```

Observed result: Terraform apply completed with no resource changes; AKS provisioning state was reported as Succeeded. This evidence is from the infrastructure pipeline run shown in the project review; it is not a substitute for rerunning validation after future code changes.

9. Glossary and reference links

Term	Meaning in this project
AKS	Azure Kubernetes Service – managed Kubernetes control plane and cluster resources.
ACR	Azure Container Registry – private registry for application container images.
IaC	Infrastructure as Code – infrastructure declared and managed through version-controlled configuration.
Terraform state	Mapping between Terraform configuration and real infrastructure resources.
Saved plan	Terraform plan output file that can be reviewed and applied later as the approved change set.
Pipeline artifact	File/package published by a pipeline run and made available to later stages.
VNet peering	Private connectivity between Azure virtual networks.
NSG	Network Security Group – rules controlling network traffic to/from associated network interfaces or subnets.

Useful official documentation

- Azure Kubernetes Service documentation
- Azure Container Registry documentation
- Terraform documentation
- Terraform AzureRM backend
- Azure Pipelines documentation
- Azure Monitor for AKS

Repository maintenance checklist

- Keep this document aligned with the actual Terraform modules and pipeline YAML whenever architecture changes.
- Update the resource inventory and architecture diagram when resources are added, renamed or removed.
- Attach only redacted pipeline evidence; never publish raw logs containing subscription IDs, email addresses, tokens or private endpoints.
- Record the date and outcome of meaningful validation runs.
- Mark future capabilities as planned until implemented and tested.

End of document

This document describes the lab state and evidence available as of 30 October 2026. Validate the repository and Azure environment before using these instructions in another subscription or for production workloads.